

Handling Execution Overruns in Hard Real-Time Control Systems Using Adaptive Scheduling

Christian Percival
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany

Abstract—Hard real-time systems require tasks to complete within strict timing constraints. In classical scheduling approaches, worst-case execution times (WCET) are used to guarantee schedulability. However, this approach often leads to poor processor utilization because many real-time tasks execute below their worst-case computation time during normal operation. This paper investigates the problem of handling execution overruns in hard real-time control systems using adaptive scheduling techniques. The work focuses on Constant Bandwidth Server (CBS) based approaches and rate adaptation methods proposed by Caccamo, Buttazzo, and Sha. The paper discusses how adaptive scheduling can improve control performance while still guaranteeing hard real-time constraints. In addition, the paper evaluates runtime overhead, resource sharing issues, and practical implementation considerations for modern embedded and control systems.

I. INTRODUCTION

Real-time systems are widely used in embedded applications such as robotics, automotive systems, industrial automation, and aerospace systems. In hard real-time systems, missing a deadline may lead to catastrophic failures or unstable system behavior. For this reason, scheduling algorithms must guarantee that all timing constraints are satisfied.

Traditional hard real-time scheduling techniques use the Worst-Case Execution Time (WCET) of tasks during schedulability analysis. Although this guarantees timing correctness, it often results in inefficient processor utilization because many tasks rarely execute for their WCET during normal operation.

Modern control applications, especially those involving image processing or visual tracking, experience highly variable execution times. For example, visual tracking algorithms may execute quickly under normal conditions but require significantly longer computation times when disturbances occur or when larger search regions must be analyzed.

To address this issue, adaptive scheduling approaches have been proposed. One important method is handling execution overruns through rate adaptation and server-based scheduling. Instead of always reserving processor time for the worst case, the scheduler dynamically adjusts task frequencies and deadlines according to actual execution demands while still guaranteeing hard deadlines.

This paper investigates techniques for handling execution overruns in hard real-time control systems, focusing primarily on the Constant Bandwidth Server (CBS) and the CBS_{hd} algorithm proposed by Caccamo et al. [1].

II. BACKGROUND

A. Hard Real-Time Systems

A hard real-time system is a system in which all tasks must complete before their deadlines. Missing deadlines may cause instability, safety hazards, or complete system failure.

Each periodic real-time task can generally be described by:

$$\tau_i(C_i, T_i, D_i)$$

where:

- C_i is the execution time,
- T_i is the task period,
- D_i is the relative deadline.

The utilization of a task is defined as:

$$U_i = \frac{C_i}{T_i}$$

The total processor utilization becomes:

$$U = \sum_{i=1}^n \frac{C_i}{T_i}$$

B. Earliest Deadline First Scheduling

Earliest Deadline First (EDF) is one of the most important dynamic priority scheduling algorithms for hard real-time systems. EDF always schedules the task with the closest deadline first.

Liu and Layland showed that EDF guarantees schedulability if:

$$U \leq 1$$

EDF provides excellent processor utilization and is widely used in adaptive real-time scheduling systems.

C. Execution Overruns

Execution overruns occur when a task exceeds its expected or average execution time. In traditional scheduling approaches, systems are designed using WCET assumptions to ensure safety.

However, this creates inefficiency because:

- average execution times are often much smaller than WCET,
- large amounts of processor bandwidth remain unused,
- control performance becomes unnecessarily limited.

Adaptive scheduling attempts to solve this problem by dynamically adjusting timing behavior according to actual execution requirements.

D. Performance Loss Index and Rate Adaptation

In adaptive real-time control systems, the control performance depends strongly on the sampling frequency of each control task. In general, higher sampling frequencies allow the controller to react faster to changes in the system, improving stability and responsiveness. However, increasing the execution frequency of tasks also increases processor utilization. Therefore, a balance must be achieved between control performance and schedulability.

To model this relationship, Caccamo et al. [1] use the Performance Loss Index (PLI), originally introduced by Seto et al. [7]. The PLI measures the degradation in control performance caused by digital sampling compared to an ideal continuous-time controller.

For each control task τ_i , the performance loss is approximated by the exponential function:

$$\Delta J_i(f_i) = \alpha_i e^{-\beta_i f_i}$$

where:

- f_i is the sampling frequency of task τ_i in Hertz (Hz),
- α_i is the magnitude coefficient,
- β_i is the decay rate,
- $\Delta J_i(f_i)$ is the resulting performance loss.

The parameter α_i represents the initial magnitude of the performance degradation at low frequencies. Larger values of α_i indicate that the control system is highly sensitive to slow sampling rates.

The parameter β_i determines how rapidly the performance improves as the sampling frequency increases. A large β_i value means that increasing the task frequency quickly reduces the performance loss.

As the task frequency increases, the exponential term decreases, causing the overall performance loss to become smaller. Therefore, higher task frequencies generally improve the quality of the control system.

The total performance loss of the complete system is defined as:

$$\Delta J(f_1, f_2, \dots, f_n) = \sum_{i=1}^n w_i \Delta J_i(f_i)$$

where w_i is a weighting factor representing the relative importance of each control task.

The objective of the rate adaptation algorithm is to minimize the total performance loss while ensuring that all tasks remain schedulable under processor utilization constraints. This allows the scheduler to dynamically adapt task frequencies according to runtime execution demands while still maintaining hard real-time guarantees.

III. RATE ADAPTATION FOR OVERRUN HANDLING

Caccamo et al. proposed a rate adaptation approach that dynamically changes task frequencies depending on runtime execution demands [1].

The key idea is:

- increase task frequency during normal execution,
- decrease frequency when overruns occur,
- maintain minimum allowable frequencies to preserve system stability.

Each task is characterized by:

$$\tau_i(WCET_i, c_i^n, f_i^{min}, \Delta J_i(f_i))$$

where:

- $WCET_i$ is the worst-case execution time,
- c_i^n is the normal execution time,
- f_i^{min} is the minimum allowed frequency,
- $\Delta J_i(f_i)$ is the performance loss index.

The performance loss index (PLI) measures the difference between ideal continuous control and sampled digital control. The paper models PLI as:

$$\Delta J_i(f_i) = \alpha_i e^{-\beta_i f_i}$$

The overall system performance loss becomes:

$$\Delta J(f_1, \dots, f_n) = \sum_i w_i \Delta J_i(f_i)$$

The objective is to minimize the total performance loss while preserving schedulability.

IV. CONSTANT BANDWIDTH SERVER

The Constant Bandwidth Server (CBS) is a scheduling mechanism designed to provide temporal isolation between tasks [3].

Each task is assigned:

- a budget Q_s ,
- a server period T_s .

The reserved bandwidth becomes:

$$U_s = \frac{Q_s}{T_s}$$

If a task exceeds its allocated budget, the server postpones its deadline. This prevents overrunning tasks from affecting the timing behavior of other tasks.

The main advantages of CBS are:

- temporal isolation,
- overload protection,
- dynamic adaptation,
- improved processor utilization.

However, the standard CBS approach may still allow hard deadline violations under certain overrun conditions.

V. THE CBSHD ALGORITHM

To solve limitations of the standard CBS algorithm, Caccamo et al. proposed an improved algorithm called CBS_{hd}.

The CBS_{hd} algorithm modifies the deadline postponement strategy by adapting the budget recharge according to the remaining computation time.

Whenever the server budget is exhausted:

- if the remaining execution time is large, the full budget is recharged,

- otherwise, only the required remaining budget is assigned.

This reduces unnecessary deadline postponements and improves responsiveness.

The CBShd algorithm maintains:

- hard real-time guarantees,
- task isolation,
- improved control performance,
- more efficient overrun handling.

Experimental results presented in the original paper show that CBShd significantly improves the performance loss index compared to traditional WCET-based scheduling.

VI. RESOURCE SHARING AND BLOCKING

In practical embedded systems, tasks often share resources such as memory buffers, communication interfaces, or hardware peripherals.

Mutual exclusion mechanisms may introduce blocking delays. To manage these effects, the Stack Resource Policy (SRP) can be used.

The blocking time must be included in schedulability analysis:

$$\sum_{i=1}^n \frac{Q_{si} + \zeta_i}{T_{si}} + \max_i \left(\frac{B_i}{T_{si}} \right) \leq 1$$

where:

- ζ_i represents critical section duration,
- B_i represents blocking time.

Including resource constraints allows adaptive scheduling methods to be applied safely in realistic embedded systems.

VII. PERFORMANCE EVALUATION

Experimental evaluations performed by Caccamo et al. demonstrate that adaptive scheduling improves system performance significantly when average execution times are lower than WCET.

The experiments showed:

- lower performance loss index,
- better processor utilization,
- improved control frequencies,
- effective handling of execution overruns.

The results also showed that smaller CBS budgets improve adaptation accuracy but increase runtime overhead because deadlines must be updated more frequently.

The measured overhead for deadline postponement operations was approximately:

$$42\mu s$$

This demonstrates an important tradeoff between scheduling precision and runtime overhead.

VIII. CONCLUSION

Handling execution overruns is a major challenge in modern hard real-time control systems. Traditional WCET-based scheduling guarantees safety but often wastes processor resources and limits control performance.

Adaptive scheduling methods such as rate adaptation and Constant Bandwidth Servers provide a more efficient solution by dynamically adjusting task timing behavior according to actual execution demands.

The CBShd algorithm improves upon traditional CBS scheduling by handling overruns more efficiently while still guaranteeing hard deadlines. Experimental results demonstrate improved processor utilization and better control performance compared to classical approaches.

These techniques are especially important for modern embedded systems involving robotics, autonomous systems, visual tracking, and advanced control applications where execution times vary significantly during runtime.

REFERENCES

- [1] M. Caccamo, G. Buttazzo, and L. Sha, "Handling Execution Overruns in Hard Real-Time Control Systems," *IEEE Transactions on Computers*, vol. 51, no. 7, pp. 835–849, 2002.
- [2] C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.
- [3] L. Abeni and G. Buttazzo, "Integrating Multimedia Applications in Hard Real-Time Systems," in *Proceedings of the IEEE Real-Time Systems Symposium*, 1998.
- [4] G. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, Springer, 2011.
- [5] T. P. Baker, "Stack-Based Scheduling of Real-Time Processes," *Real-Time Systems*, vol. 3, no. 1, pp. 67–99, 1991.
- [6] G. Buttazzo, G. Lipari, M. Caccamo, and L. Abeni, "Elastic Scheduling for Flexible Workload Management," *IEEE Transactions on Computers*, vol. 51, no. 3, pp. 289–302, 2002.
- [7] D. Seto, J. P. Lehoczky, L. Sha, and K. G. Shin, "On Task Schedulability in Real-Time Control Systems," in *Proceedings of the IEEE Real-Time Systems Symposium (RTSS)*, 1996.